

Código Haskell — Paradigma Funcional

Explicação didática linha a linha — funcional/Main.hs

[Folha de Pagamento](#)

[Paradigmas](#)

[Linha a linha](#)

Como ler este código

Haskell modela o problema com dados imutáveis e funções puras. A folha é uma transformação de uma lista de funcionários em resultados e agregados.

Regras de negócio usadas

- $\text{valor_hora} = \text{salário_bruto} / 180$
- $\text{extras} = \text{valor_hora} \times 1,5 \times \text{horas extras}$
- $\text{base_irrf} = \text{bruto} + \text{extras} - \text{INSS} - \text{dependentes} \times 189,59$
- $\text{líquido} = \text{bruto} + \text{extras} - \text{INSS} - \text{IRRF}$
- Os mesmos três funcionários são usados em todas as versões: Ana, Bruno e Carla.

Visão geral da solução

A implementação produz os mesmos resultados verificados nas quatro linguagens: Total da folha = 6564.90; maior líquido = Ana (3317.07); menor líquido = Carla (1245.83).

Explicação linha a linha

Linha	Código	Explicação didática
1	<code>module Main where</code>	Declara o módulo principal executável.
2		Linha em branco para separar definições independentes.
3	<code>import Data.List (maximumBy, minimumBy)</code>	Importa funções para encontrar maior e menor elemento por critério.
4	<code>import Data.Ord (comparing)</code>	Importa função auxiliar para comparar usando um campo específico.
5		Linha em branco para separar definições independentes.
6	<code>data Funcionario = Funcionario</code>	Define o tipo de dado Funcionario com campos nomeados.
7	<code>{ nome :: String</code>	Campo do registro Funcionario, representando uma entrada do problema.

Linha	Código	Explicação didática
8	<code>, salarioBruto :: Double</code>	Campo do registro Funcionario, representando uma entrada do problema.
9	<code>, dependentes :: Int</code>	Campo do registro Funcionario, representando uma entrada do problema.
10	<code>, horasExtras :: Int</code>	Campo do registro Funcionario, representando uma entrada do problema.
11	<code>} deriving (Show)</code>	Permite exibir o valor automaticamente em depuração.
12		Linha em branco para separar definições independentes.
13	<code>data ResultadoFolha = ResultadoFolha</code>	Define o tipo que guarda o resultado calculado.
14	<code>{ resultadoNome :: String</code>	Campo do ResultadoFolha, armazenando uma parte do cálculo.
15	<code>, bruto :: Double</code>	Campo do ResultadoFolha, armazenando uma parte do cálculo.
16	<code>, extras :: Double</code>	Campo do ResultadoFolha, armazenando uma parte do cálculo.
17	<code>, inss :: Double</code>	Campo do ResultadoFolha, armazenando uma parte do cálculo.
18	<code>, irrf :: Double</code>	Campo do ResultadoFolha, armazenando uma parte do cálculo.
19	<code>, liquido :: Double</code>	Campo do ResultadoFolha, armazenando uma parte do cálculo.
20	<code>} deriving (Show)</code>	Permite exibir o valor automaticamente em depuração.
21		Linha em branco para separar definições independentes.
22	<code>type Faixa = (Double, Double)</code>	Cria sinônimo: uma faixa é um par (limite, taxa).
23		Linha em branco para separar definições independentes.
24	<code>faixasINSS :: [Faixa]</code>	Assinatura de tipo da lista de faixas de INSS.

Linha	Código	Explicação didática
25	<code>faixasINSS =</code>	Inicia a lista de faixas do INSS.
26	<code>[(1412.00, 0.075)</code>	Item da lista de faixas: limite e alíquota.
27	<code>, (2666.00, 0.090)</code>	Item da lista de faixas: limite e alíquota.
28	<code>, (4000.00, 0.120)</code>	Item da lista de faixas: limite e alíquota.
29	<code>, (1 / 0, 0.140)</code>	Item da lista de faixas: limite e alíquota.
30	<code>]</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
31		Linha em branco para separar definições independentes.
32	<code>faixasIRRF :: [Faixa]</code>	Assinatura de tipo da lista de faixas de IRRF.
33	<code>faixasIRRF =</code>	Inicia a lista de faixas do IRRF.
34	<code>[(2259.00, 0.000)</code>	Item da lista de faixas: limite e alíquota.
35	<code>, (2826.00, 0.075)</code>	Item da lista de faixas: limite e alíquota.
36	<code>, (3751.00, 0.150)</code>	Item da lista de faixas: limite e alíquota.
37	<code>, (4664.00, 0.225)</code>	Item da lista de faixas: limite e alíquota.
38	<code>, (1 / 0, 0.275)</code>	Item da lista de faixas: limite e alíquota.
39	<code>]</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
40		Linha em branco para separar definições independentes.
41	<code>aliquota :: [Faixa] -> Double -> Double</code>	Assinatura: recebe lista de faixas e valor, devolve a taxa.
42	<code>aliquota [] _ = error "Tabela de faixas vazia ou valor fora das faixas"</code>	Caso base: falha se a tabela estiver vazia ou incompleta.
43	<code>aliquota ((limite, taxa):resto) valor</code>	Desestrutura a primeira faixa e o restante da lista.
44	<code> valor <= limite = taxa</code>	Guarda: se o valor cabe no limite, esta é a taxa.

Linha	Código	Explicação didática
45	otherwise = aliquota resto valor	Caso contrário, procura recursivamente no restante da lista.
46		Linha em branco para separar definições independentes.
47	valorExtras :: Funcionario -> Double	Assinatura da função pura que calcula horas extras.
48	valorExtras f = (salarioBruto f / 180.0) * 1.5 * fromIntegral (horasExtras f)	Calcula extras usando salário bruto, divisor 180, adicional 1,5 e horas.
49		Linha em branco para separar definições independentes.
50	deducaoDependentes :: Funcionario -> Double	Assinatura da função que calcula dedução por dependentes.
51	deducaoDependentes f = fromIntegral (dependentes f) * 189.59	Converte dependentes para Double e multiplica por 189,59.
52		Linha em branco para separar definições independentes.
53	calcularFuncionario :: Funcionario -> ResultadoFolha	Assinatura da função principal de cálculo individual.
54	calcularFuncionario f = ResultadoFolha (nome f) brutoFuncionario extrasFuncionario valorINSS valorIRRF valorLiquido	Constrói o ResultadoFolha a partir dos valores calculados no where.
55	where	Inicia definições locais usadas somente nesta função.
56	brutoFuncionario = salarioBruto f	Nome local para o salário bruto do funcionário.
57	extrasFuncionario = valorExtras f	Nome local para o valor das horas extras.
58	valorINSS = brutoFuncionario * aliquota faixasINSS brutoFuncionario	Nome local para o salário bruto do funcionário.
59	baseIRRF = max 0.0 (brutoFuncionario + extrasFuncionario - valorINSS - deducaoDependentes f)	Nome local para o salário bruto do funcionário.
60	valorIRRF = baseIRRF * aliquota faixasIRRF baseIRRF	Calcula base do IRRF e aplica mínimo zero com max.

Linha	Código	Explicação didática
61	<code>valorLiquido = brutoFuncionario + extrasFuncionario - valorINSS - valorIRRF</code>	Nome local para o salário bruto do funcionário.
62		Linha em branco para separar definições independentes.
63	<code>formatarResultado :: ResultadoFolha -> String</code>	Assinatura da função que transforma resultado em texto.
64	<code>formatarResultado r = resultadoNome r ++</code>	Monta a string de saída concatenando campos formatados.
65	<code>" Bruto: " ++ moeda (bruto r) ++</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
66	<code>" Extras: " ++ moeda (extras r) ++</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
67	<code>" INSS: " ++ moeda (inss r) ++</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
68	<code>" IRRF: " ++ moeda (irrf r) ++</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
69	<code>" Liquido: " ++ moeda (liquido r)</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
70		Linha em branco para separar definições independentes.
71	<code>moeda :: Double -> String</code>	Assinatura da função de formatação monetária.
72	<code>moeda = formatDouble 2</code>	Especializa formatDouble para duas casas decimais.
73		Linha em branco para separar definições independentes.
74	<code>formatDouble :: Int -> Double -> String</code>	Assinatura da função genérica de arredondamento/formatação.

Linha	Código	Explicação didática
75	<code>formatDouble casas valor = showFFloatFixed casas valor</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
76		Linha em branco para separar definições independentes.
77	<code>showFFloatFixed :: Int -> Double -> String</code>	Assinatura da função que monta a string decimal manualmente.
78	<code>showFFloatFixed casas valor = let fator = 10 ^ casas :: Integer</code>	Calcula potência de 10 usada para arredondar casas decimais.
79	<code>arredondado = round (valor * fromIntegral fator) :: Integer</code>	Arredonda o valor escalado para inteiro.
80	<code>inteiro = arredondado `div` fator</code>	Arredonda o valor escalado para inteiro.
81	<code>decimal = arredondado `mod` fator</code>	Arredonda o valor escalado para inteiro.
82	<code>dec = show decimal</code>	Separa a parte decimal.
83	<code>in show inteiro ++ "." ++ replicate (casas - length dec) '0' ++ dec</code>	Separa a parte inteira.
84		Linha em branco para separar definições independentes.
85	<code>funcionarios :: [Funcionario]</code>	Assinatura da lista fixa de funcionários.
86	<code>funcionarios =</code>	Inicia a lista de funcionários de teste.
87	<code>[Funcionario "Ana" 4500.00 2 8</code>	Item da lista: nome, salário bruto, dependentes e horas extras.
88	<code>, Funcionario "Bruno" 2200.00 0 0</code>	Item da lista: nome, salário bruto, dependentes e horas extras.
89	<code>, Funcionario "Carla" 1300.00 1 4</code>	Item da lista: nome, salário bruto, dependentes e horas extras.
90	<code>]</code>	Linha de Haskell que mantém a solução como composição de dados imutáveis e funções puras.
91		Linha em branco para separar definições independentes.

Linha	Código	Explicação didática
92	<code>main :: IO ()</code>	Assinatura da ação principal de entrada/saída.
93	<code>main = do</code>	Inicia bloco de ações no mundo IO.
94	<code>let resultados = map calcularFuncionario funcionarios</code>	Calcula todos os resultados aplicando map.
95	<code>totalFolha = sum (map liquido resultados)</code>	Soma os salários líquidos com sum e map.
96	<code>maior = maximumBy (comparing liquido) resultados</code>	Importa funções para encontrar maior e menor elemento por critério.
97	<code>menor = minimumBy (comparing liquido) resultados</code>	Importa função auxiliar para comparar usando um campo específico.
98	<code>putStrLn "Folha de pagamento - paradigma funcional (Haskell)"</code>	Imprime uma linha de texto no terminal.
99	<code>putStrLn "Politica: valores monetarios exibidos com 2 casas decimais; calculos internos usam Double.\n"</code>	Imprime uma linha de texto no terminal.
100	<code>mapM_ (putStrLn . formatarResultado) resultados</code>	Imprime uma linha de texto no terminal.
101	<code>putStrLn ("\nTotal da folha: " ++ moeda totalFolha)</code>	Soma os salários líquidos com sum e map.
102	<code>putStrLn ("Maior salario liquido: " ++ resultadoNome maior ++ " (" ++ moeda (liquido maior) ++ ")")</code>	Imprime uma linha de texto no terminal.
103	<code>putStrLn ("Menor salario liquido: " ++ resultadoNome menor ++ " (" ++ moeda (liquido menor) ++ ")")</code>	Imprime uma linha de texto no terminal.

Resumo para apresentação oral

Explique que Haskell evita mutação: cada função recebe dados e devolve novos valores. map transforma a lista de funcionários em resultados; sum, maximumBy e minimumBy fazem os agregados.